

Mechanistic Interpretability

CS60216: Safety Fundamentals for Generative AI

March 20th, 2025

Interpretability (in AI)

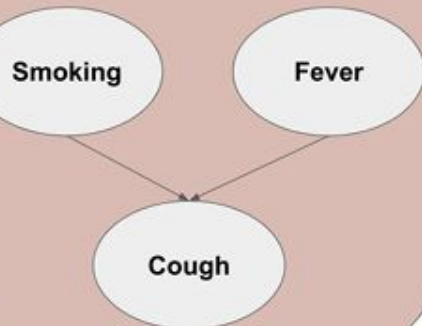
Definition: *The study of understanding the decisions that AI systems make and putting them into easily human-understandable terms.*



Why: to use that understanding to iteratively better design systems that are more *performant* and *human-understandable*.

Historically models were small

BAYES NETS



LINEAR REGRESSION

$$y = mx + b$$

MULTIVARIATE LINEAR REGRESSION

$$y = Wx + b$$

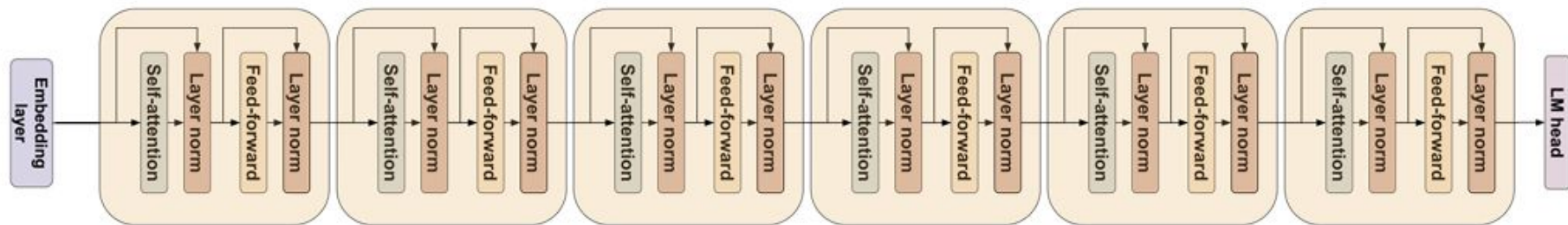
LOGISTIC REGRESSION

$$\text{sigmoid}(Wx + b)$$

2 LAYER MLP w/ ReLU

$$W(\text{ReLU}(Wx + b)) + c$$

How do we make sense of this huge model?



First attempt: Probing

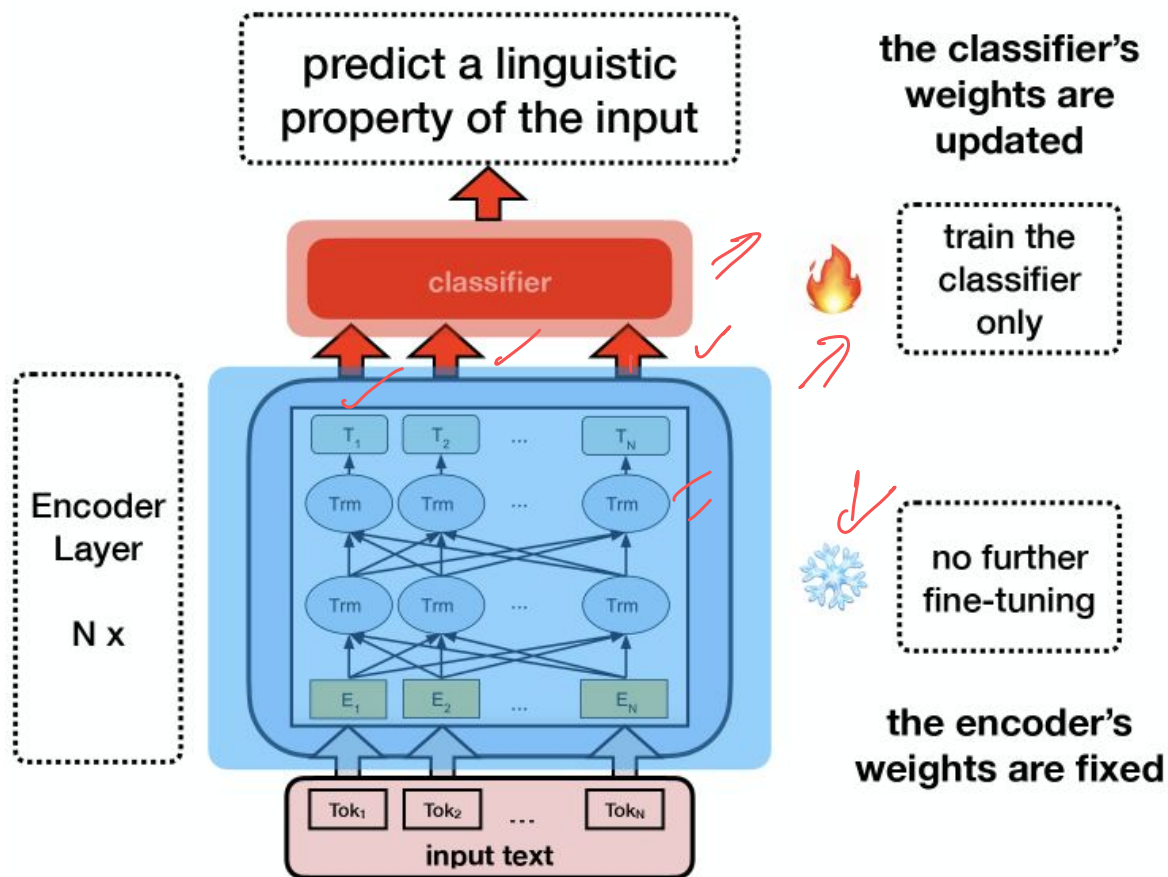
1? → class

Definition: *A classifier that is specifically trained to predict some property from a pretrained model's representations.*

✓
How? Given a pretrained model, we use the representations it produces to train a classifier (without further fine-tuning the model) to predict a linguistic property of the input text

↗

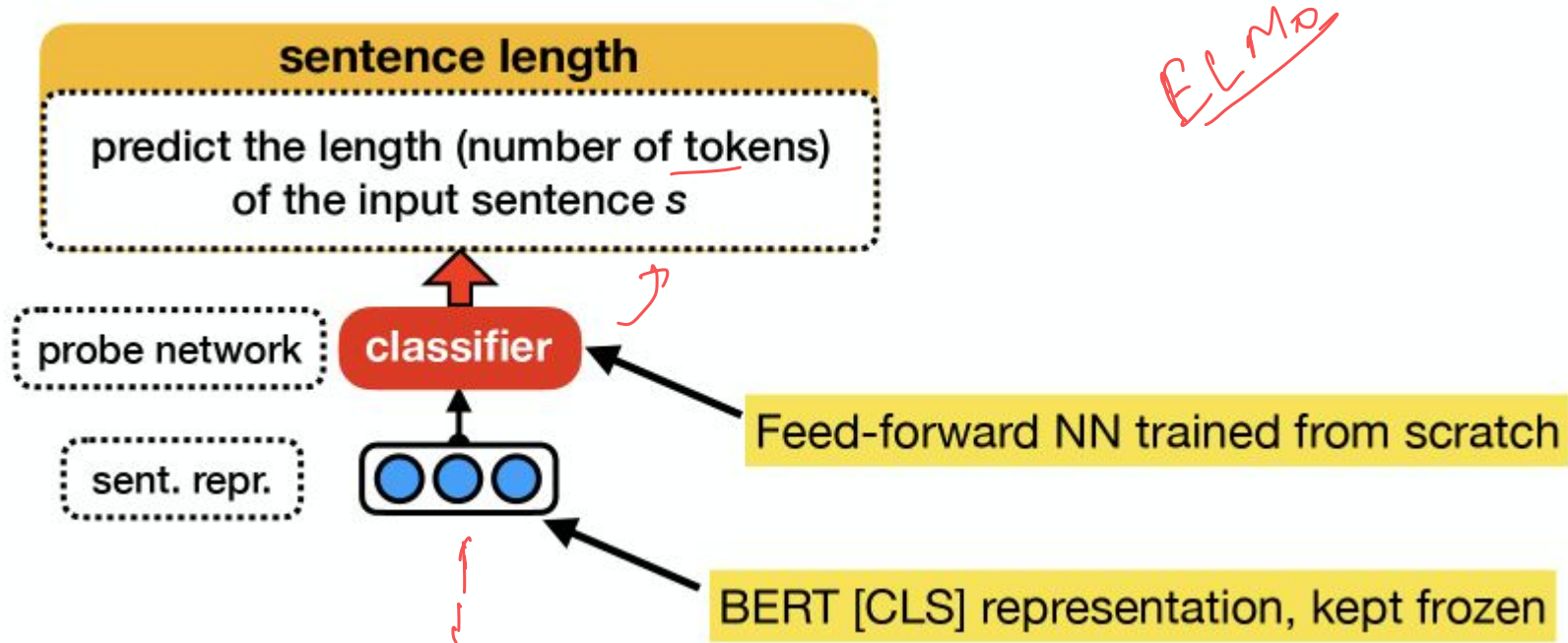
Probe Approach



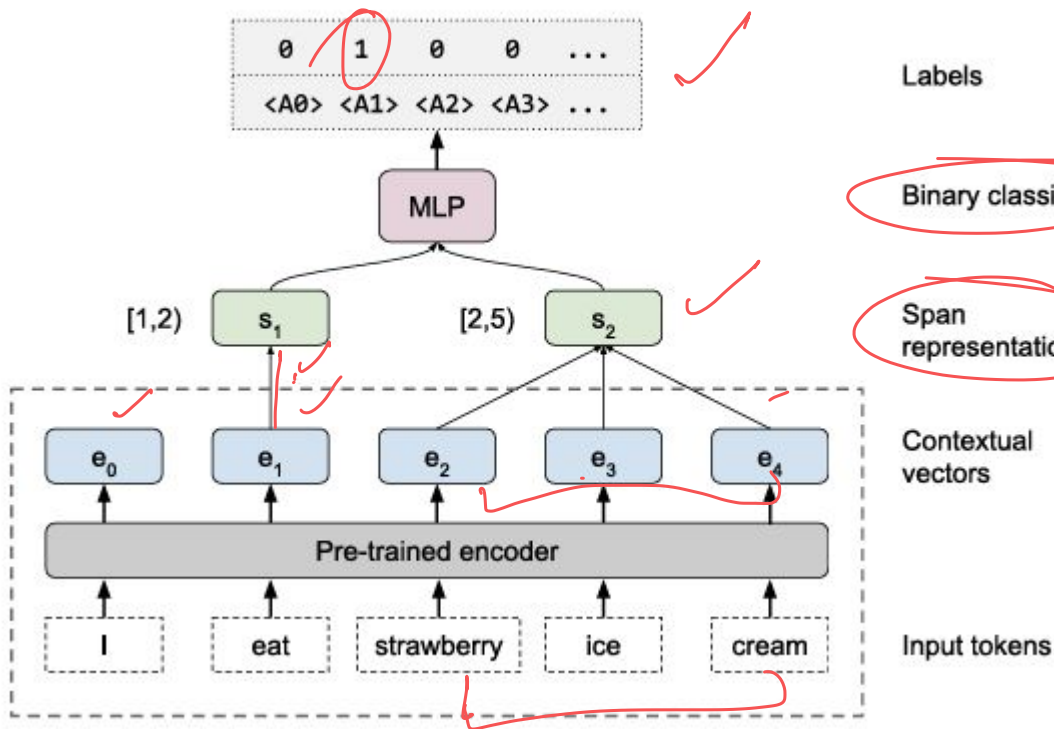
Motivation of probe tasks

- if we can train a classifier to predict a property of the input text based on its representation, it means the property is encoded somewhere in the representation
- if we cannot train a classifier to predict a property of the input text based on its representation, it means the property is not encoded in the representation or not encoded in a useful way, considering how the representation is likely to be used

Probing



Edge Probing



All parameters inside the dashed line are fixed, while we train the span pooling and MLP classifiers to extract information from the contextual vectors. The example shown is for semantic role labeling, where $s(1) = [12)$ corresponds to the predicate (“eat”), while $s(2) = [25)$ is the argument (“strawberry ice cream”), and we predict label $A1$ as positive and others as negative. For entity and constituent labeling, only a single span is used.

Edge Probing

POS	The important thing about Disney is that it is a global [brand] ₁ . → NN (Noun)
Constit.	The important thing about Disney is that it [is a global brand] ₁ . → VP (Verb Phrase)
Depend.	[Atmosphere] ₁ is always [fun] ₂ → nsubj (nominal subject)
Entities	The important thing about [Disney] ₁ is that it is a global brand. → Organization
SRL	[The important thing about Disney] ₂ [is] ₁ that it is a global brand. → Arg1 (Agent)
SPR	[It] ₁ [endorsed] ₂ the White House strategy. . . → {awareness, existed_after, . . . }
Coref. ^O	The important thing about [Disney] ₁ is that [it] ₂ is a global brand. → True
Coref. ^W	[Characters] ₂ entertain audiences because [they] ₁ want people to be happy. → True Characters entertain [audiences] ₂ because [they] ₁ want people to be happy. → False
Rel.	The [burst] ₁ has been caused by water hammer [pressure] ₂ . → Cause-Effect(e_2, e_1)

How to combine the representations across layers?

Lexical Baselines. For GPT and for BERT, we use the learned subword embeddings from the full model.

Contextual vectors: For BERT and GPT, we compare two methods:

cat where we concatenate the subword embeddings with the activations of the top layer, and

mix where we take a linear combination of layer activations (including embeddings) using learned task-specific scalars, similar to ELMo.

$$\text{ELMo}_k^{\text{task}} = E(R_k; \Theta^{\text{task}}) = \gamma^{\text{task}} \sum_{j=0}^L s_j^{\text{task}} \mathbf{h}_{k,j}^{LM}$$

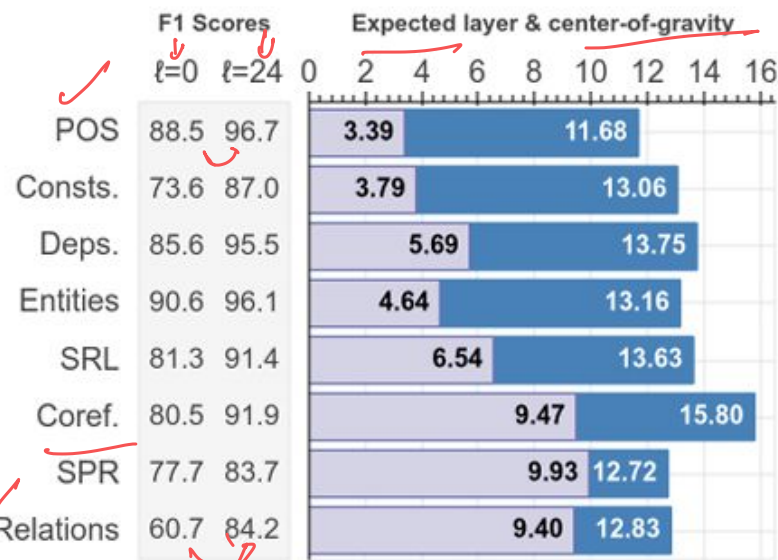
last layer (pointing to L)
emb (pointing to s_j^{task})

Performance Evaluation

	BERT-base				BERT-large				
	F1 Score			Abs. Δ ELMo	F1 Score			Abs. Δ (base) ELMo	
	Lex.	cat	mix		Lex.	cat	mix		
Part-of-Speech	88.4	97.0	96.7	0.0	88.1	96.5	96.9	0.2	0.2
Constituents	68.4	83.7	86.7	2.1	69.0	80.1	87.0	0.4	2.5
Dependencies	80.1	93.0	95.1	1.1	80.2	91.5	95.4	0.3	1.4
Entities	90.9	96.1	96.2	0.6	91.8	96.2	96.5	0.3	0.9
SRL (all)	75.4	89.4	91.3	1.2	76.5	88.2	92.3	1.0	2.2
Core roles	74.9	91.4	93.6	1.0	76.3	89.9	94.6	1.0	2.0
Non-core roles	76.4	84.7	85.9	1.8	76.9	84.1	86.9	1.0	2.8
OntoNotes coref.	74.9	88.7	90.2	6.3	75.7	89.6	91.4	1.2	7.4
SPR1	79.2	84.7	86.1	1.3	79.6	85.1	85.8	-0.3	1.0
SPR2	81.7	83.0	83.8	0.7	81.6	83.2	84.1	0.3	1.0
Winograd coref.	54.3	53.6	54.9	1.4	53.0	53.8	61.4	6.5	7.8
Rel. (SemEval)	57.4	78.3	82.0	4.2	56.2	77.6	82.4	0.5	4.6
Macro Average	75.1	84.8	86.3	1.9	75.2	84.2	87.3	1.0	2.9

ELMo-style
scalar mixing
(mix) instead of
concatenation
improves
performance
significantly

BERT rediscovers the NLP pipeline (Tenney et al. 2019)



24

POS - part of speech tagging (e.g. this word is a noun)

consts - constituent labeling (e.g. is this span a noun phrase)

deps - dependency labeling (e.g. is span_one the subject and span_two the object)

entities - named entity labeling (e.g. this word is a person)

SRL - semantic role labeling (what roles are the spans playing with each other: "Mary (pusher) pushed John (pushee)")

coref - coreference (do span_one and span_two refer to the same entity or event)

SPR - semantic proto-role (identifying attributes like awareness so is Mary aware that they are doing the pushing)

relations - relation classification (predicting the real-world relation between two spans given a set of these)

Figure 1: Summary statistics on BERT-large. Columns on left show F1 dev-set scores for the baseline ($P_{\tau}^{(0)}$) and full-model ($P_{\tau}^{(L)}$) probes. Dark (blue) are the mixing weight center of gravity (Eq. 2); light (purple) are the expected layer from the cumulative scores (Eq. 4).

BERT rediscovers the NLP pipeline (Tenney et al. 2019)

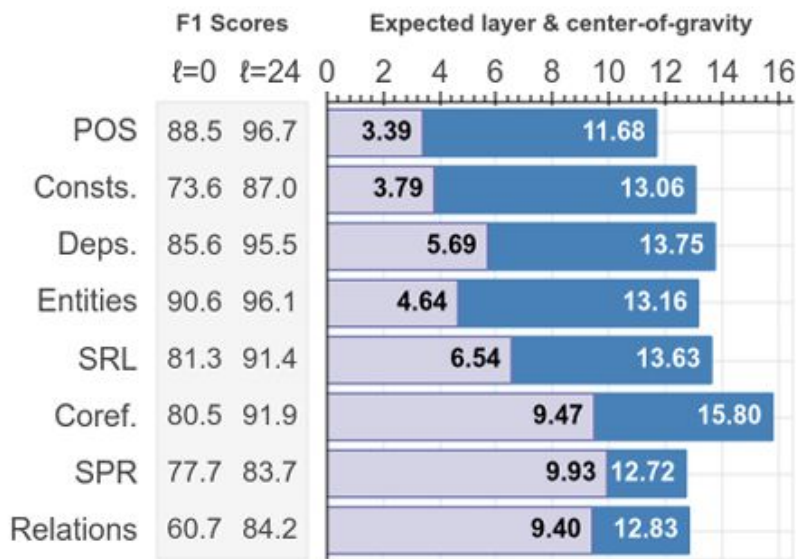


Figure 1: Summary statistics on BERT-large. Columns on left show F1 dev-set scores for the baseline ($P_{\tau}^{(0)}$) and full-model ($P_{\tau}^{(L)}$) probes. Dark (blue) are the mixing weight center of gravity (Eq. 2); light (purple) are the expected layer from the cumulative scores (Eq. 4).

Representation for the probe:

$$\mathbf{h}_{i,\tau} = \gamma_{\tau} \sum_{\ell=0}^L s_{\tau}^{(\ell)} \mathbf{h}_i^{(\ell)}$$

Handwritten notes: 0.05 , 0.2 , 0.3 (learnable weights)

We learn these weights jointly with the probing classifier in order to allow it to extract information from the many layers of an encoder without adding a large number of parameters.

After the probing model is trained, we extract the learned coefficients in order to estimate the contribution of different layers to that particular task.

BERT rediscovers the NLP pipeline (Tenney et al. 2019)

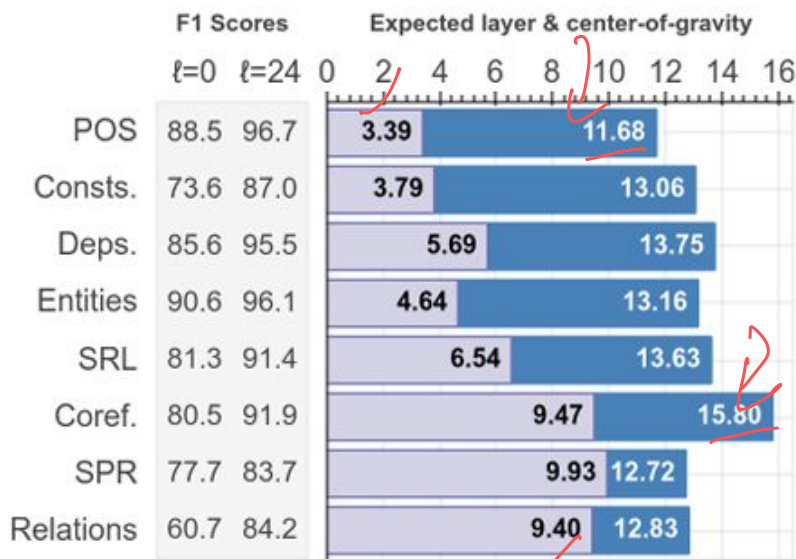


Figure 1: Summary statistics on BERT-large. Columns on left show F1 dev-set scores for the baseline ($P_{\tau}^{(0)}$) and full-model ($P_{\tau}^{(L)}$) probes. Dark (blue) are the mixing weight center of gravity (Eq. 2); light (purple) are the expected layer from the cumulative scores (Eq. 4).

the expected layer at which the probing model correctly labels an example

a higher center-of-gravity means that the information needed for that task is captured by higher layers

Center of gravity:

$$\bar{E}_s[\ell] = \sum_{\ell=0}^L \ell \cdot s_{\tau}^{(\ell)}$$

BERT rediscovers the NLP pipeline (Tenney et al. 2019)

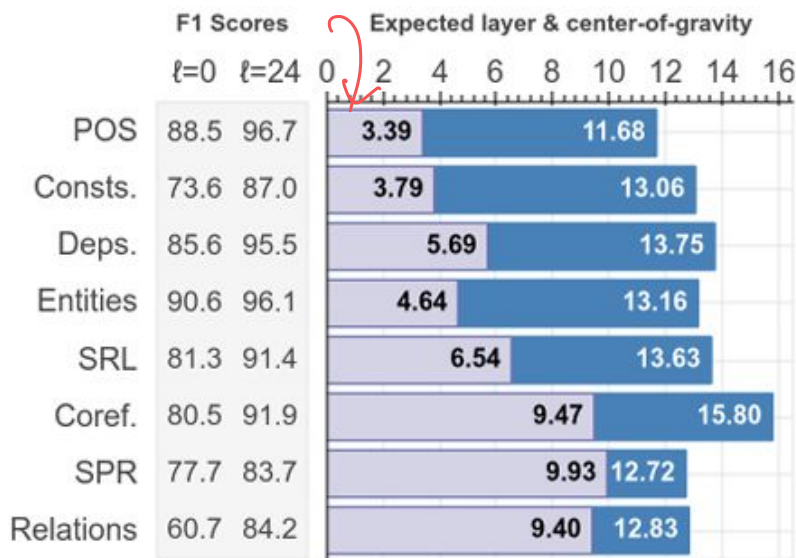


Figure 1: Summary statistics on BERT-large. Columns on left show F1 dev-set scores for the baseline ($P_{\tau}^{(0)}$) and full-model ($P_{\tau}^{(L)}$) probes. Dark (blue) are the mixing weight center of gravity (Eq. 2); light (purple) are the expected layer from the cumulative scores (Eq. 4).

$$\Delta_{\tau}^{(\ell)} = \text{Score}(\underline{P_{\tau}^{(\ell)}}) - \text{Score}(P_{\tau}^{(\ell-1)})$$

measures how much better we do on the probing task if we observe one additional encoder layer ℓ :

$$\bar{E}_{\Delta}[\ell] = \frac{\sum_{\ell=1}^L \ell \cdot \Delta_{\tau}^{(\ell)}}{\sum_{\ell=1}^L \Delta_{\tau}^{(\ell)}}$$

This can be thought of as, approximately, the expected layer at which the probing model correctly labels an example, assuming that example is resolved at some layer $\ell \geq 1$ of the encoder.

BERT rediscovers the NLP pipeline (Tenney et al. 2019)

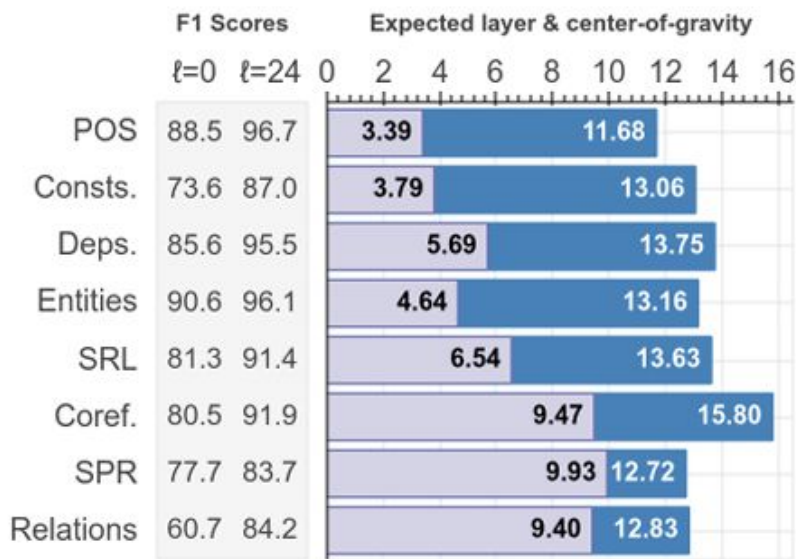
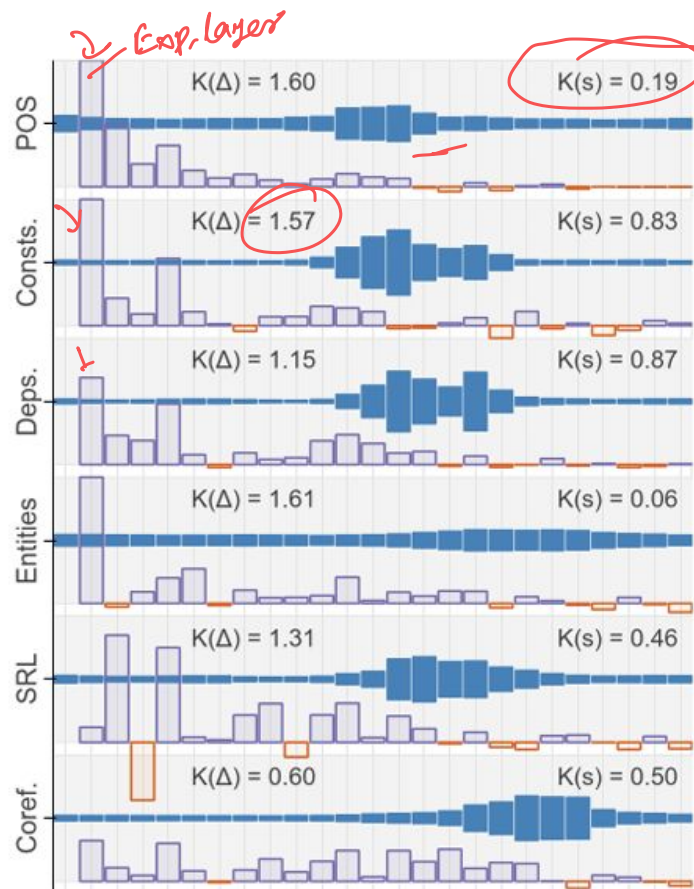


Figure 1: Summary statistics on BERT-large. Columns on left show F1 dev-set scores for the baseline ($P_{\tau}^{(0)}$) and full-model ($P_{\tau}^{(L)}$) probes. Dark (blue) are the mixing weight center of gravity (Eq. 2); light (purple) are the expected layer from the cumulative scores (Eq. 4).



$$K(\star) = \text{KL}(\star || \text{Uniform})$$

Gain insights into the latent knowledge of the model

<https://arxiv.org/pdf/2212.03827>

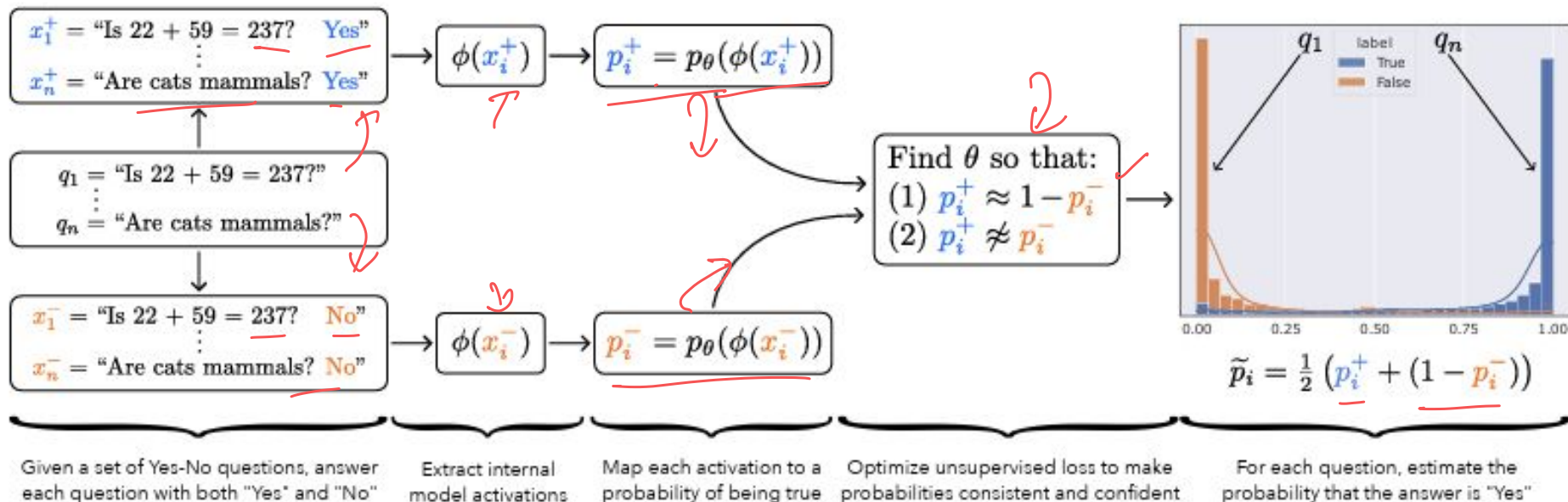
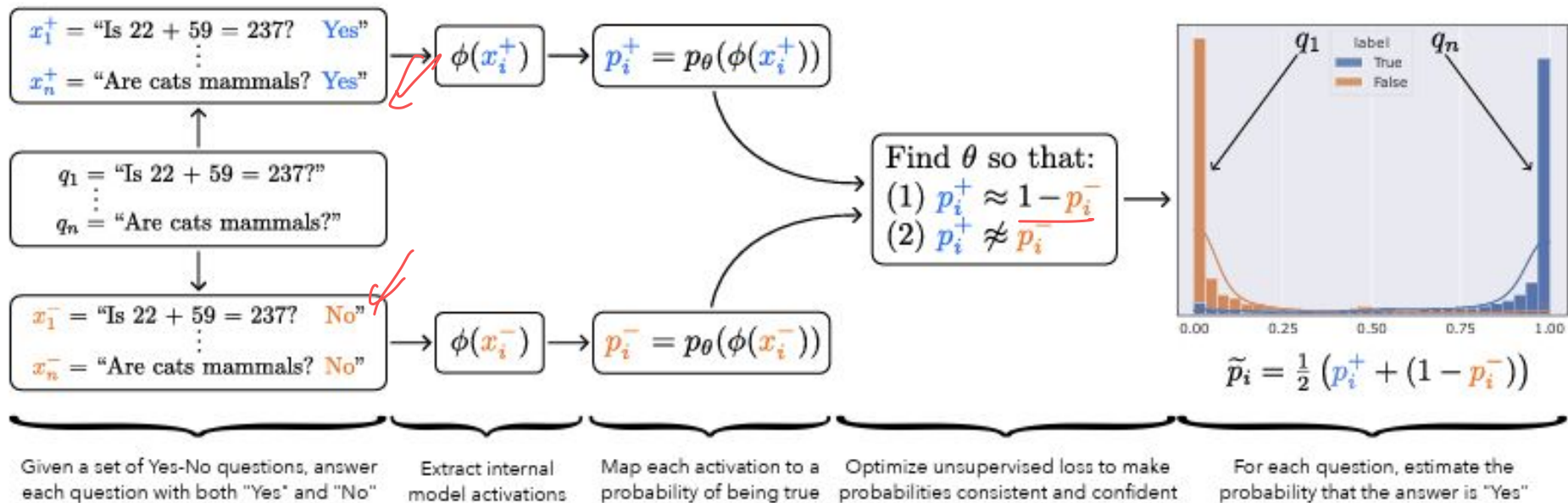


Figure 1: An illustration of our method, Contrast-Consistent Search (CCS). For each yes-no question q_i , we let x_i^+ and x_i^- be the natural language statements where we answer q_i as "Yes" and "No" respectively. Answering the question q_i then amounts to determining which of x_i^+ or x_i^- is true.

Our method uses no labels and no model outputs, but still learns to accurately answers questions.

Gain insights into the latent knowledge of the model

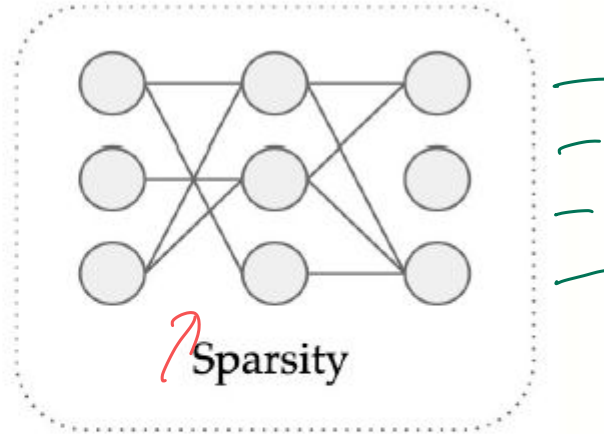
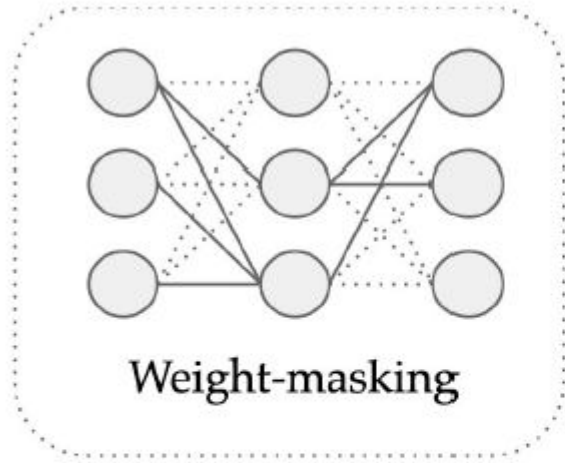
<https://arxiv.org/pdf/2212.03827>



$$L_{\text{consistency}}(\theta, b; q_i) := [p_{\theta, b}(x_i^+) - (1 - p_{\theta, b}(x_i^-))]^2 \quad L_{\text{confidence}}(\theta, b; q_i) := \min\{p_{\theta, b}(x_i^+), p_{\theta, b}(x_i^-)\}^2$$

$$L_{\text{CCS}}(\theta, b) := \frac{1}{n} \sum_{i=1}^n L_{\text{consistency}}(\theta, b; q_i) + L_{\text{confidence}}(\theta, b; q_i)$$

Inner interpretability methods



E.g., replacing last layer with a sparse layer and debug model behavior by simply inspecting the few relevant features

Sparse Linear Layers

Let (X, y) be the standardized data matrix (mean zero and variance one) and output. respectively.

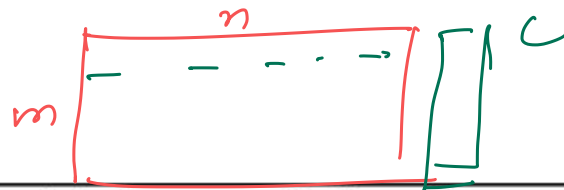
In our setting, X corresponds to the (normalized) deep feature representations of input data points, while y is the target.

Our goal is to fit a sparse linear model of the form $\mathbb{E}(Y|X = x) = x^T \beta + \beta_0$

Optimization problem: $\min_{\beta} \frac{1}{2N} \|X^T \beta + \beta_0 - y\|_2^2 + \lambda R_{\alpha}(\beta)$

$$R_{\alpha}(\beta) = (1 - \alpha) \frac{1}{2} \|\beta\|_2^2 + \alpha \|\beta\|_1$$

Explaining LMs



Algorithm 2 Word cloud feature visualization for language models for a vocabulary V , a corpus w_{ij} for $i, j \in [m] \times [n]$ of m sentences with n words

```
1: for  $i = 1 \dots m$  do
2:    $\beta_i = \text{LIME}(w_i)$  // generate LIME explanation for each sentence
3: end for
4: for  $w \in V$  do
5:    $K_w = \sum_{ij:w=w_{ij}} 1$  // count number of occurrences of word
6:    $\hat{\beta}_w = \frac{1}{K_w} \sum_{ij:w=w_{ij}} \beta_{ij}$  // calculate average LIME explanation of word
7: end for
8: return Wordcloud( $\beta, V$ ) // generate word cloud for vocabulary  $V$  weighted by  $\beta$ 
```



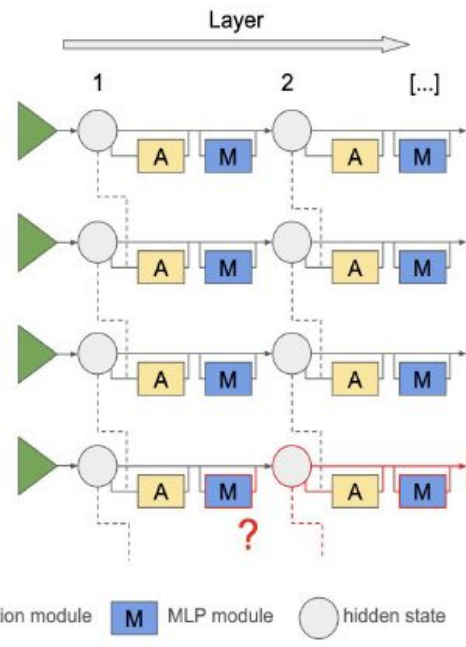
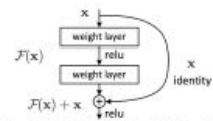
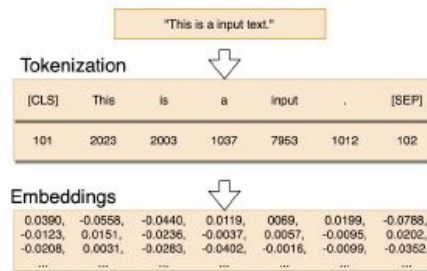
word clouds visualizing the positive and negative activations

Logit Lens

Goal: Study how transformer prediction changes in a forward pass (Or how individual transformer modules at a layer i affect outcome)

"Logit lens":

- Use unembedding projection (usually after the last layer) at intermediate state or layer



Logit Lens on GPT-2

- Input and output
 - As *input*, GPT takes a sequence of tokens. Each token is a single item from a vocabulary of $N_v=50257$ byte pairs (mostly English words).
 - As *output*, GPT returns a probability distribution over the vocabulary. It is trained so this distribution predicts the next token.
 - That is, the model's outputs are shifted forward by one position relative to the inputs. The token at position i should, after flowing through the layers of the model, turn into the token at position $i+1$. (More accurately, a distribution over the token at position $i+1$.)
- Vocab and embedding spaces
 - The vocab has size $N_v=50257$, but GPT works internally in a smaller "embedding" vector space, of dimension N_e .
 - For example, in the GPT-2 1558M model size, $N_e=1600$. (Below, I'll often assume we're talking about GPT-2 1558M for concreteness.)
 - There is an N_v -by- N_e embedding matrix W which is used to project the vocab space into the embedding space and vice versa.

<https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens>

Logit Lens on GPT-2

- In, blocks, out
 - The first thing that happens to the inputs is a multiplication by W , which projects them into the embedding space. [1]
 - The resulting 1600-dimensional vector then passes through many neural network blocks, each of which returns another 1600-dimensional vector.
 - At the end, the final 1600-dimensional vector is multiplied by W 's transpose to project back into vocab space.
 - The resulting 50257-dim vectors are treated as logits. Applying the softmax function to them gives you the output probability distribution.

<https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens>

To summarize

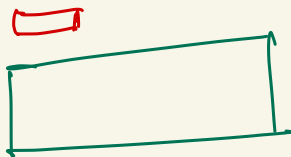
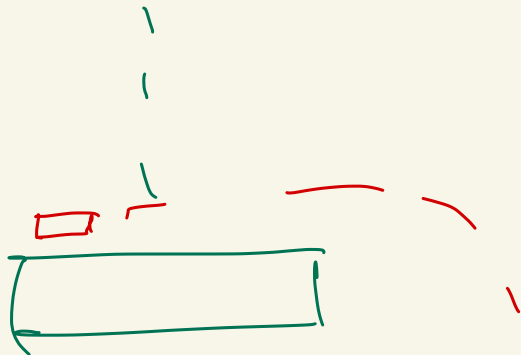
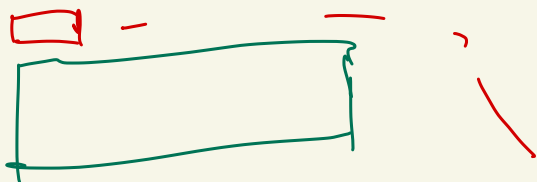
- Project the input tokens from vocab space into the 1600-dim embedding space
- Modify this 1600-dim vector many times
- Project the final 1600-dim vector back into vocab space

Logit Lens: Basic idea

We have a "dictionary," W , that lets us convert between vocab space and embedding space at any point. We know that *some* vectors in embedding space make sense when converted into vocab space:

- The very first embedding vectors are just the input tokens (in embedding space)
- The very last embedding vectors are just the output logits (in embedding space)

What about the 1600-dim vectors produced in the middle of the network, say the output of the 12th layer or the 33rd? If we convert them to vocab space, do the results make sense? The answer is *yes*.



	h_out	h46_out	h44_out	h42_out	h40_out	h38_out	h36_out	h34_out	h32_out	h30_out	h28_out	h26_out	h24_out	h22_out	h20_out	h18_out	h16_out	h14_out	h12_out	h10_out	h8_out	h6_out	h4_out	h2_out	h0_out																																																																																																																																																	
	'.'	'we'	'show'	'a'	'AN'	'models'	'based'	'models'	'a'	'N'	'targeted'	'we'	'found'	'a'	'rap'	's'	'based'	'nd'	'which'	'hybrid'	'targeted'	'we'	'found'	'naive'	'FP'	'Ms'	'based'	'nd'	'which'	'ambitious'	'targeted'	'we'	'found'	'algorithms'	'FP'	's'	'based'	'nd'	'which'	'widely'	'targeted'	'we'	'found'	'camp'	'FP'	's'	'based'	'nd'	'which'	'widely'	'targeted'	'although'	'found'	'algorithm'	'FP'	'ouch'	'based'	'nd'	'000'	'single'	'h18_out	'targeted'	'although'	'focus'	'camp'	'AP'	'ouch'	'based'	'nd'	'000'	'single'	'h16_out	'targeted'	'unlike'	'focus'	'camp'	'MP'	'IME'	'based'	'nd'	'000'	'single'	'h14_out	'targeted'	'note'	'target'	'camp'	'MS'	'IME'	'based'	'nd'	'000'	'single'	'h12_out	'target'	'unlike'	'hope'	'split'	'MP'	'ouch'	'based'	'nd'	'000'	'massive'	'h10_out	'updated'	'however'	'd'	'session'	'tott'	'IME'	'style'	'nd'	'000'	'massive'	'h8_out	'target'	'however'	'target'	'evaluation'	'rom'	'IME'	'based'	'nd'	'000'	'enormous'	'h6_out	'focused'	'however'	'd'	'ees'	'rou'	'ools'	'based'	'nd'	'which'	'enormous'	'h4_out	'target'	'however'	'can'	'ees'	'rou'	'ools'	'sided'	'nd'	'and'	'enormous'	'h2_out	'guid'	'and'	'Hardy'	'ees'	'rou'	'ools'	'based'	'nd'	'and'	'enormous'	'h0_out	'chini'	'and'	'd'	'train'	'rou'	'ools'	'based'	'nd'	'and'	'isolated'

Model's top-token and its logit





Model's top-token and its logit



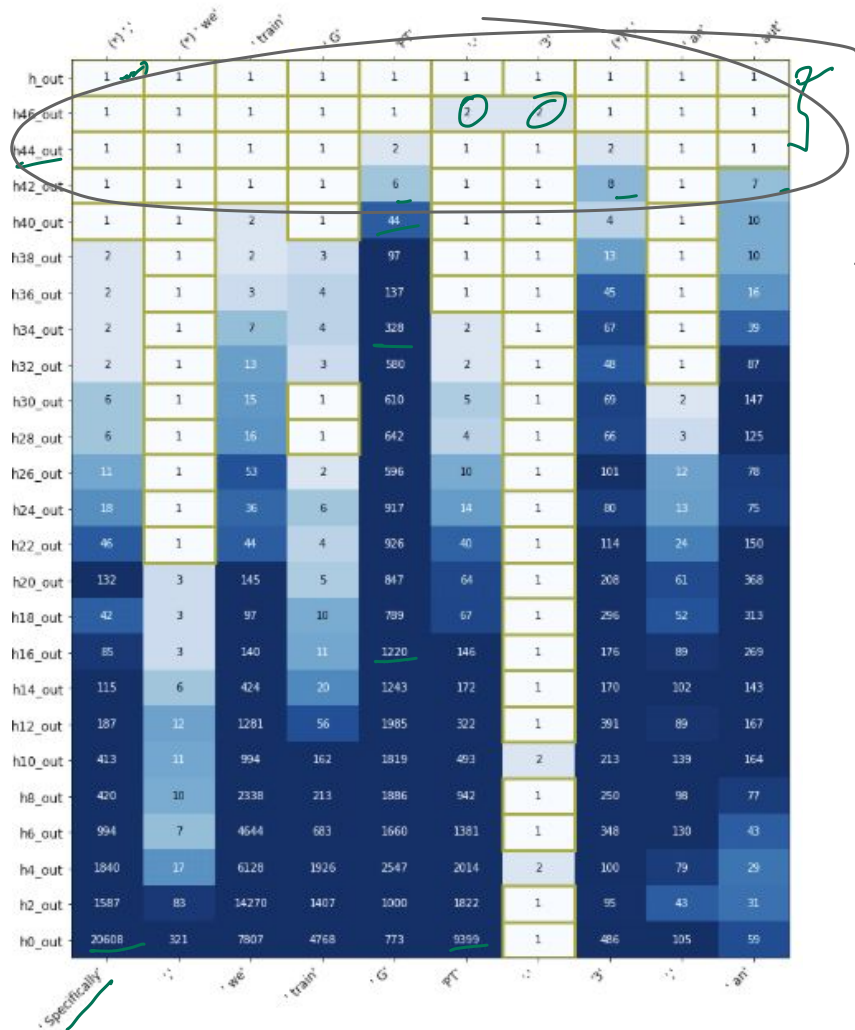
Looking at the ranks

The view till now focuses only on the top-1 guess at each layer, which is a reductive window on the full distributions.

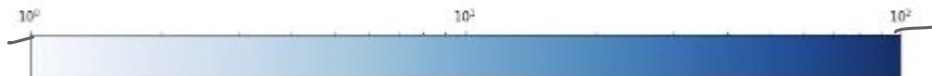
Another way to look at things: we still reduce the *final* output to the top-1 guess, but we compare other distributions to the final one by looking at the rank of the final top-1 guess.

Even if the middle of the model hasn't yet converged to the final answer, maybe it's got that answer somewhere in its top 3, top 10, etc. That's a lot better than "top 50257."

(Remember: these are ranks of *the model's final top-1 prediction*, not *the true token*.)



model's top token and its rank over the ~50K vocab



KL divergence

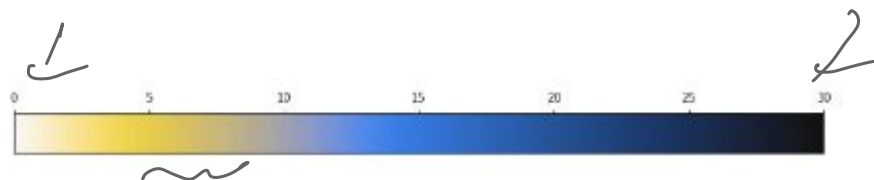
Another way of comparing the similarity of two probability distributions is the KL divergence.

Taking the KL divergence of the intermediate probabilities w/r/t the final probabilities, we get a more continuous view of how the distributions smoothly converge to the model's output.

Because KL divergence is a more holistic measure of the similarity between two distributions than the ones I've used above, it's also my preferred metric for making the point that *nothing looks like the input*.

	(*)	(*) we	'train'	'G'	'PT'	'G'	'G'	(*)	'an'	'aut'
h_out	'.'	'we'	'show'	'a'	'AN'	'models'	'based'	'.'	'a'	'N'
h46_out	'.'	'we'	'show'	'a'	'AN'	'.'	'.'	'.'	'a'	'N'
h44_out	'.'	'we'	'show'	'a'	'RM'	'models'	'based'	'models'	'a'	'N'
h42_out	'.'	'we'	'show'	'a'	'rams'	'models'	'based'	'models'	'a'	'algorithm'
h40_out	'.'	'we'	'demonstrate'	'a'	'machine'	'models'	'based'	'models'	'a'	'algorithm'
h38_out	'we'	'we'	'demonstrate'	'neural'	'rap'	'models'	'based'	'models'	'a'	'algorithm'
h36_out	'we'	'we'	'demonstrate'	'neural'	'rap'	'models'	'based'	'models'	'a'	'algorithm'
h34_out	'we'	'we'	'demonstrate'	'models'	'rap'	'model'	'based'	'models'	'a'	'algorithm'
h32_out	'we'	'we'	'simulated'	'models'	'rap'	'model'	'based'	'models'	'a'	'adaptive'
h30_out	'targeted'	'we'	'found'	'a'	'rap'	'.'	'based'	'.'	'which'	'hybrid'
h28_out	'targeted'	'we'	'found'	'a'	'FP'	'Ms'	'based'	'rd'	'which'	'ambitious'
h26_out	'targeted'	'we'	'found'	'naive'	'FP'	'Ms'	'based'	'rd'	'which'	'ambitious'
h24_out	'targeted'	'we'	'found'	'algorithms'	'FP'	's'	'based'	'rd'	'which'	'widely'
h22_out	'targeted'	'we'	'found'	'camp'	'FP'	's'	'based'	'rd'	'which'	'widely'
h20_out	'targeted'	'although'	'found'	'algorithm'	'FP'	'ouch'	'based'	'rd'	'000'	'single'
h18_out	'targeted'	'although'	'focus'	'camp'	'AP'	'ouch'	'based'	'rd'	'000'	'single'
h16_out	'targeted'	'unlike'	'focus'	'camp'	'MP'	'INE'	'based'	'rd'	'000'	'single'
h14_out	'targeted'	'note'	'target'	'camp'	'MS'	'INE'	'based'	'rd'	'000'	'single'
h12_out	'target'	'unlike'	'hope'	'split'	'MP'	'ouch'	'based'	'rd'	'000'	'massive'
h10_out	'updated'	'however'	'd'	'session'	'lott'	'INE'	'style'	'rd'	'000'	'massive'
h8_out	'target'	'however'	'target'	'evaluation'	'ram'	'INE'	'based'	'rd'	'000'	'enormous'
h6_out	'focused'	'however'	'd'	'ees'	'you'	'tools'	'based'	'rd'	'which'	'enormous'
h4_out	'target'	'however'	'can'	'ees'	'you'	'tools'	'sided'	'rd'	'and'	'enormous'
h2_out	'guk'	'and'	'hardy'	'ees'	'you'	'tools'	'based'	'rd'	'and'	'enormous'
h0_out	'chul'	'and'	'd'	'train'	'you'	'tools'	'based'	'rd'	'and'	'isolated'
h_in	'Specifically'	'.'	'we'	'train'	'G'	'PT'	'G'	'G'	'.'	'an'

kl divergence from output distributions



Copying a rare text

Sometimes it's clear that the next token should be a "copy" of an earlier token: whatever arbitrary thing was in that slot, spit it out again.

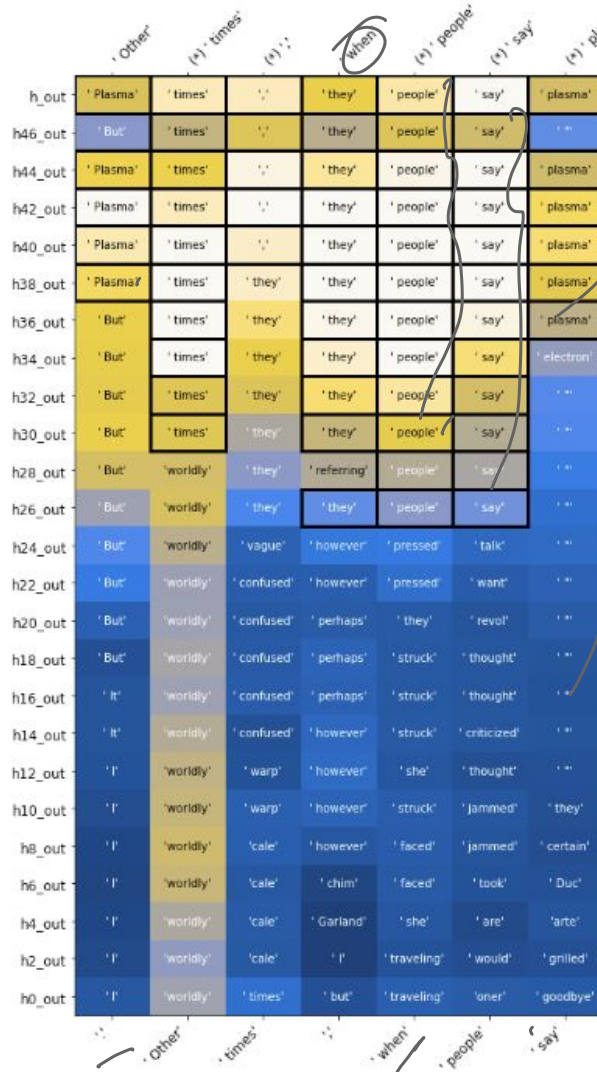


If this is a token with relatively low prior probability, one would think it would be useful to "keep it around" from the input so later positions can look at it and copy it. But as we saw, the input is never "kept around"!

What happens instead? I tried this text:

Sometimes, when people say plasma, they mean a state of matter. Other times, when people say plasma





Apparently it is keeping around a representation of the token "plasma" with enough resolution to copy it . . . but it only retrieves this representation at the end! (In the rank view, the rank of plasma is quite low until the very end.)

The repetition is directly visible in the input: "when people say" is copied verbatim. If you just applied the rule "if input seems to be repeating, keep repeating it," you'd be good. Instead, the model scrambles away the pattern, then recovers it later through some other computational route.

Tuned Lens

<https://arxiv.org/pdf/2303.08112>

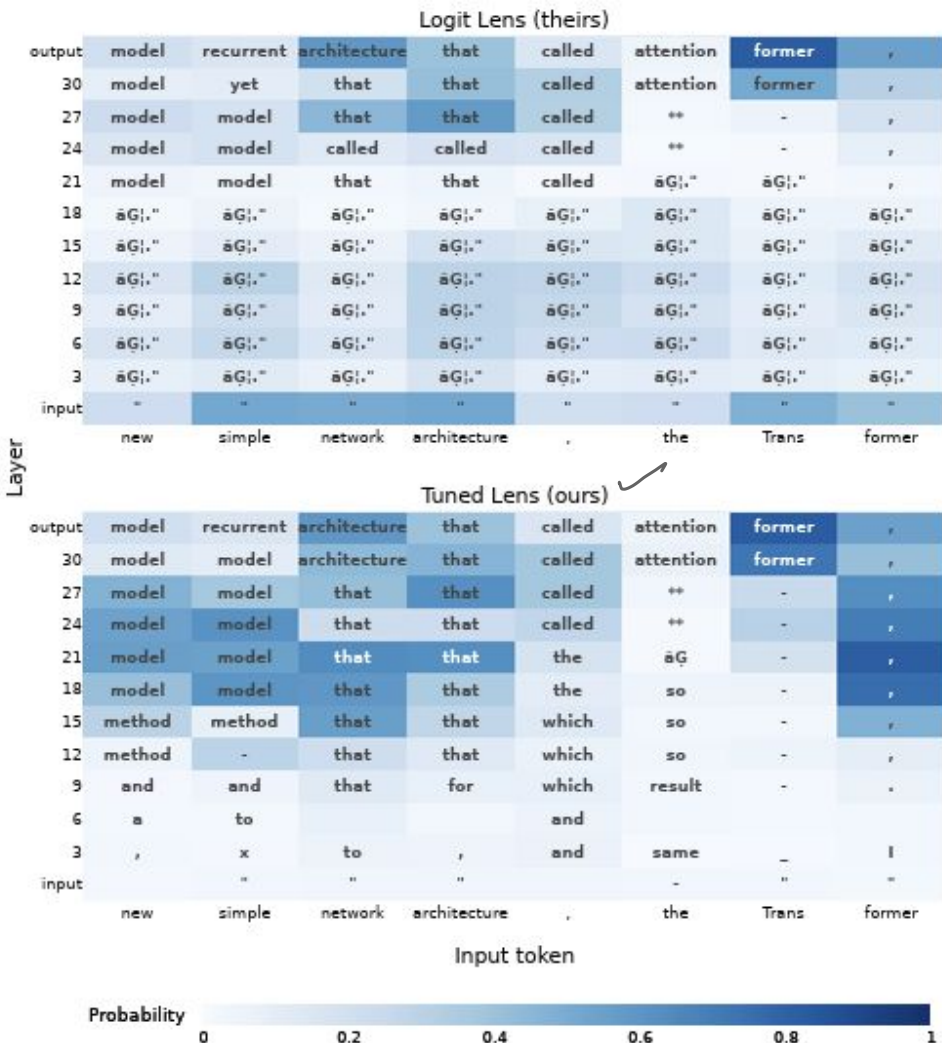
Figure 1. Comparison of our method, the *tuned lens* (bottom), with the “logit lens” (top) for GPT-Neo-2.7B prompted with an excerpt from the abstract of Vaswani et al. (2017). Each cell shows the top-1 token predicted by the model at the given layer and token index. The logit lens fails to elicit interpretable predictions before layer 21, but our method succeeds.

$$\text{TunedLens}_\ell(\mathbf{h}_\ell) = \text{LogitLens}(\underbrace{A_\ell}_{dx^d} \underbrace{\mathbf{h}_\ell}_d + \underbrace{\mathbf{b}_\ell}_d)$$

We refer to $(A_\ell, \mathbf{b}_\ell)$ as the *translator* for layer ℓ .

Loss function. We train the translators to minimize KL between the tuned lens logits and the final layer logits:

$$\operatorname{argmin}_x \mathbb{E} \left[D_{KL}(f_{>\ell}(\mathbf{h}_\ell) \parallel \text{TunedLens}_k(\mathbf{h}_\ell)) \right]$$



Why does in-context learning work?

In-context learning means language models learning to

- do new tasks
 - better predict tokens, or
 - generally reduce their loss
- 
- during the forward-pass at inference-time,
 - without any gradient-based updates to the model's parameters.
-

A Mathematical Framework for Transformer Circuits

AUTHORS

Nelson Elhage^{*,†}, Neel Nanda^{*}, Catherine Olsson^{*}, Tom Henighan[†], Nicholas Joseph[†],
Ben Mann[†], Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova DasSarma,
Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Andy Jones,
Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark,
Jared Kaplan, Sam McCandlish, Chris Olah^{*}

^{*} Core Research Contributor; [†] Core Infrastructure Contributor; ^{*} Correspondence to colah@anthropic.com;
Author contributions statement below.

AFFILIATION

Anthropic

PUBLISHED

Dec 22, 2021

<https://transformer-circuits.pub/2021/framework/index.html>

Induction Head Hypothesis

prefix

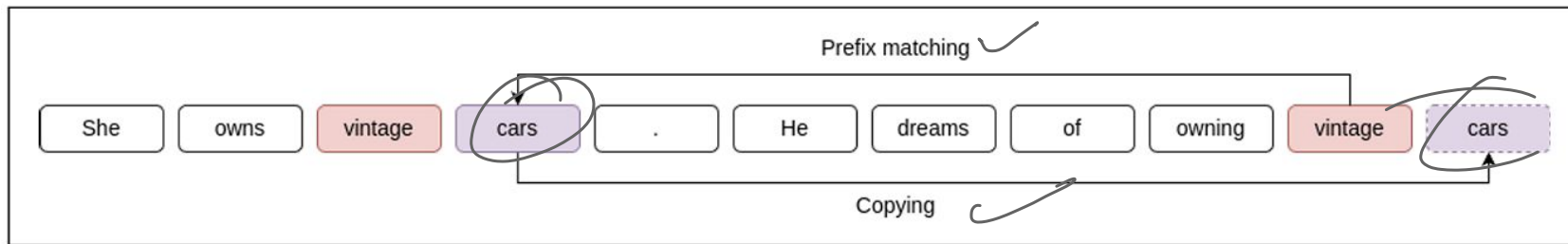


Figure 12.3 An induction head looking at *vintage* uses the *prefix matching* mechanism to find a prior instance of *vintage*, and the *copying* mechanism to predict that *cars* will occur again. Figure from [Crosbie and Shutova \(2022\)](#).

The function of the induction head is to predict repeated sequences.

For example if it sees the pattern $A B \dots A$ in an input sequence, it predicts that B will follow, instantiating the pattern completion rule A
 $B \dots A B$.

Induction head: the formal definition

Formally, **we define an induction head** as one which exhibits the following two properties on a repeated random sequence of tokens:

- **Prefix matching**: The head attends back to previous tokens that were followed by the current and/or recent tokens. That is, it attends to the token which induction would suggest comes next.
- **Copying**: The head's output increases the logit corresponding to the attended-to token.

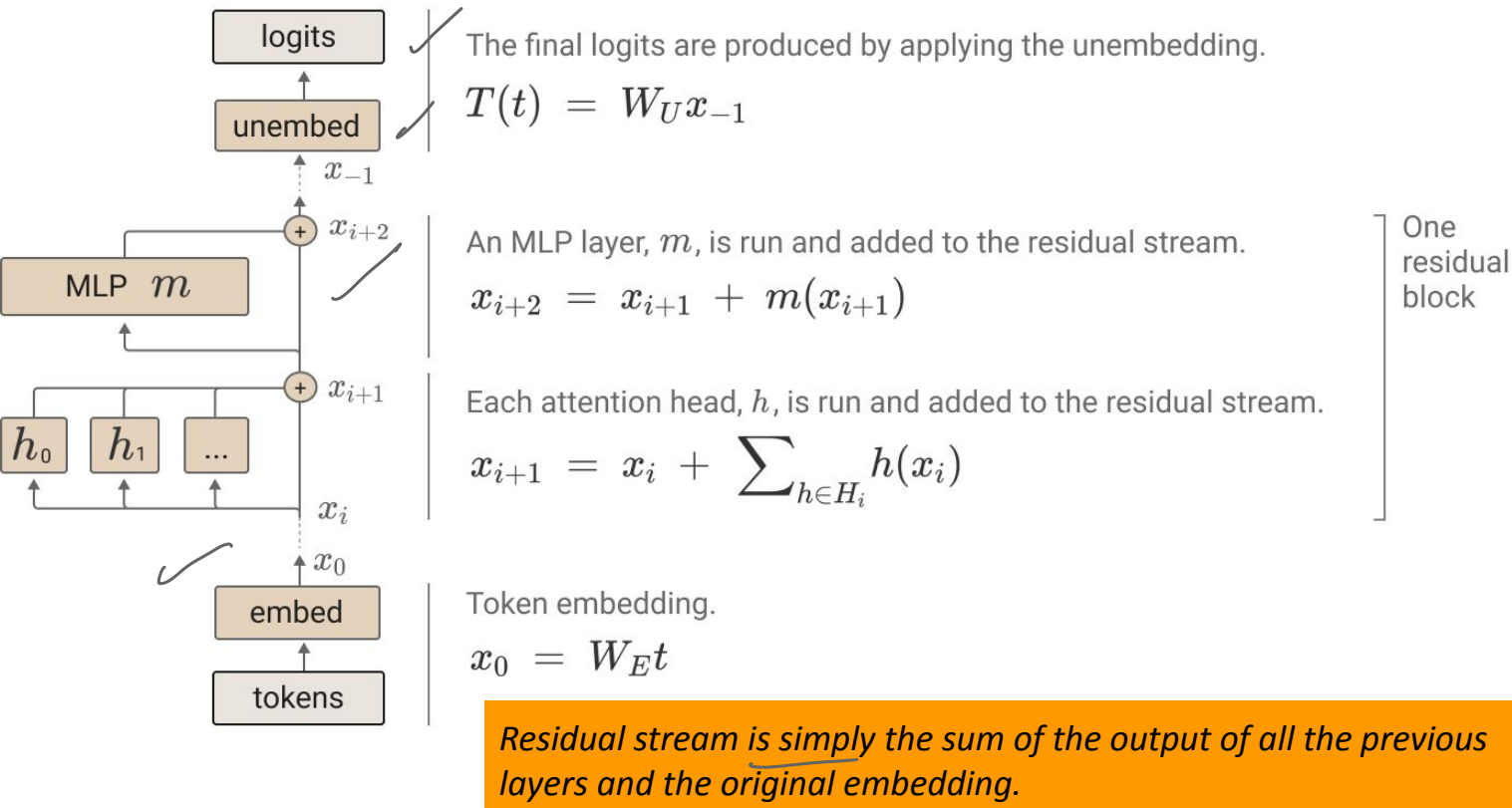
In other words, induction heads are any heads that empirically increase the likelihood of [B] given [A][B]...[A] when shown a repeated sequence of completely random tokens. An illustration of induction heads' behavior is shown here:



Induction Head Hypothesis

- Induction heads are the name for a circuit, which is a kind of abstract component of a network.
- The induction head circuit is part of the attention computation in transformers, discovered by looking at mini language models with only 1-2 attention heads.

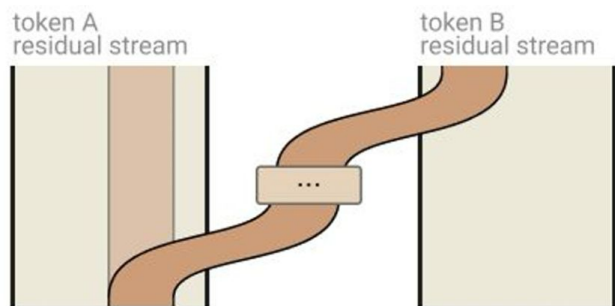
Transformer Architecture



The residual stream

Attention heads move information across residual streams.

MLPs move information across dimensions within a residual stream.



Attention heads copy information from the residual stream of one token to the residual stream of another. They typically write to a different subspace than they read from.

Important notions

- The central object in the transformer is the **residual stream**.
- Different heads in each layer can be thought of as **operating independently** of each other, reading and writing into the residual stream.
- Heads can compose to form **circuits**. For instance, **K -composition** is when the output of one head is used to generate the **key vector** in the attention calculations of a subsequent head.
- We can describe the weight matrices $\underline{W_Q}$, $\underline{W_K}$ and $\underline{W_V}$ as **reading from** (or **projecting from**) the residual stream, and $\underline{W_O}$ as **writing to** (or **embedding into**) the residual stream.

Important notions

$$(W_K x_i)^T W_Q x_j \quad z_i^T \underbrace{W_K^T W_Q}_{W} x_j$$

- We can think of the combined operations W_Q and W_K in terms of a single, low-rank matrix $W_{QK} := W_Q W_K^T$, called the **QK circuit**.
 - This matrix defines a bilinear form on the vectors in the residual stream: $v_i^T W_{QK} v_j$ is the attention paid by the i th token to the j th token.
 - Conceptually, this matrix tells us which tokens information is moved to & from in the residual stream.
- We can think of the combined operations W_V and W_O in terms of a single matrix $W_{OV} := W_V W_O$, called the **OV circuit**.
 - This matrix defines a map from residual stream vectors to residual stream vectors: if v_j is the residual stream vector at the source token, then $v_j^T W_{OV}$ is the vector that gets moved from token j to the destination token (if j is attended to).
 - Conceptually, this matrix tells us what information is moved from a token, if that token is attended to.

Subspaces in the residual stream

As information moves through this model, it will be stored in 4 different subspaces.

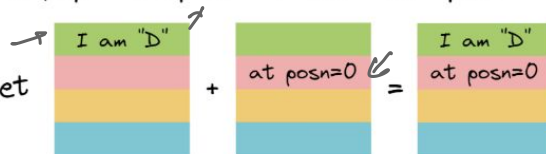
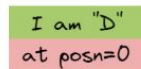
Each one has a different interpretation, so it will help to describe the subspaces in these terms:



To make the notation cleaner, I'll leave subspaces empty if they've not been used yet.

e.g. after adding token & positional encodings, we get

which I'll write as just



More notation: "A" is a one-hot encoded token vector, i.e. $\text{"A"}^T W_E = \text{I am "A"}$ ✓

n is for a one-hot encoded position vector, i.e. $n^T W_{pos} = \text{at posn=n}$ ✓

An example: Embedding

Embedding



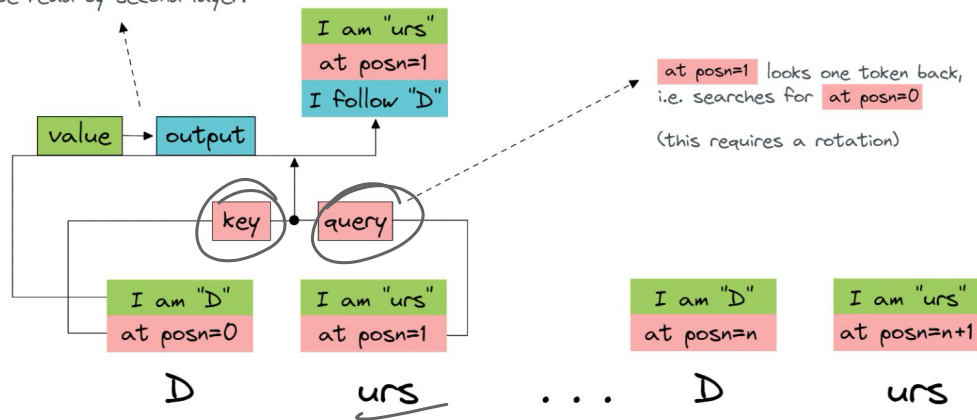
token embedding, from W_E
positional embedding, from W_{pos}

D urs ... D urs ...

Layer 0 attention head

Summary: each token looks one position backwards, and gets the information about which token preceded it.

"I am 'D'" is converted to "I follow 'D'" and moved to the destination token, ready to be read by second layer.



QK circuit

Query vector is $\begin{bmatrix} \text{I am "urs"} \\ \text{at posn=1} \end{bmatrix}^T W_Q = \text{"I'm looking for posn } 1 - 1 = 0"$

Key vector is $\begin{bmatrix} \text{I am "D"} \\ \text{at posn=0} \end{bmatrix}^T W_K = \text{"I'm at posn } 0"$

i.e. we subtract one from the current position (this requires a rotation!)

The attention score (the dot product of the key and query vectors) is large, because the key is a good match for the query.

QK circuit

Query vector is $\begin{bmatrix} \text{I am "urs"} \\ \text{at posn=1} \end{bmatrix}^T W_Q = \text{"I'm looking for posn } 1 - 1 = 0"$

Key vector is $\begin{bmatrix} \text{I am "D"} \\ \text{at posn=0} \end{bmatrix}^T W_K = \text{"I'm at posn } 0"$

i.e. we subtract one from the

OV circuit

The OV circuit reads token information and moves it to a different subspace (so it doesn't erase the embedding information which is already stored at the destination token).

We have:

The OV circuit reads token information, and moves it to a different subspace (so it doesn't erase the token embedding information which is already stored at the destination token).

We have:

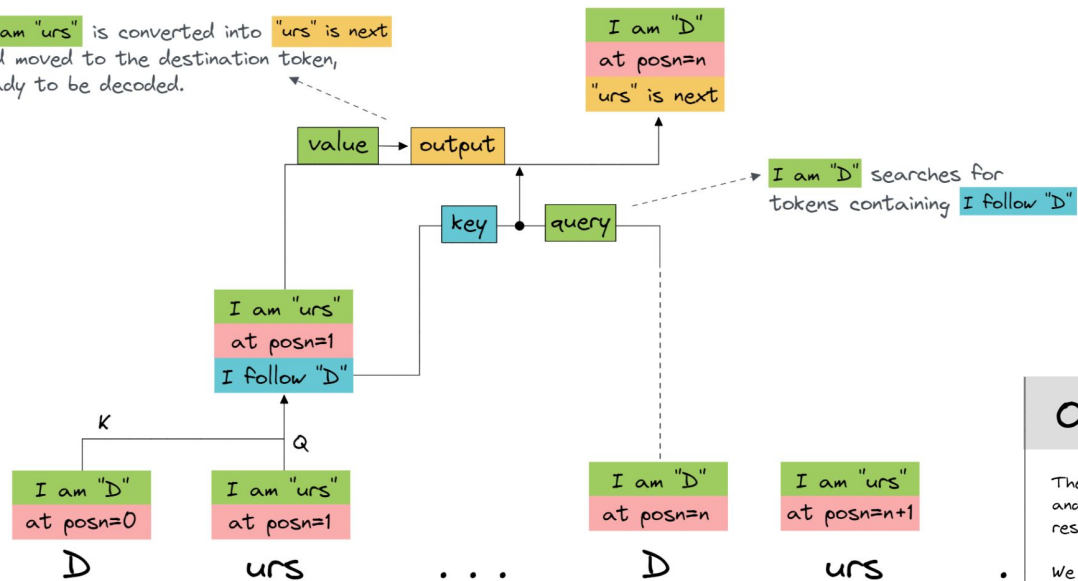
$\begin{bmatrix} \text{I am "D"} \\ \text{at posn=0} \end{bmatrix}^T W_V W_O = \text{I follow "D"}$

which gets added to the residual stream for the first "urs" token.

Layer 1 attention head

Summary: the 2nd "D" token looks back for the token following the 1st "D" (which is "urs"), and uses that as its prediction.

"I am 'urs'" is converted into "urs" is next and moved to the destination token, ready to be decoded.



QK circuit

The query is $\begin{bmatrix} \text{I am "D"} \\ \text{at posn=n} \end{bmatrix}^T W_Q = \text{"I'm looking for a token following "D"}$

The key is $\begin{bmatrix} \text{I am "urs"} \\ \text{at posn=1} \\ \text{I follow "D"} \end{bmatrix}^T W_K = \text{"I follow "D"}$

The attention score (the dot product of the key and query vectors) is large, because the key is a good match for the query.

OV circuit

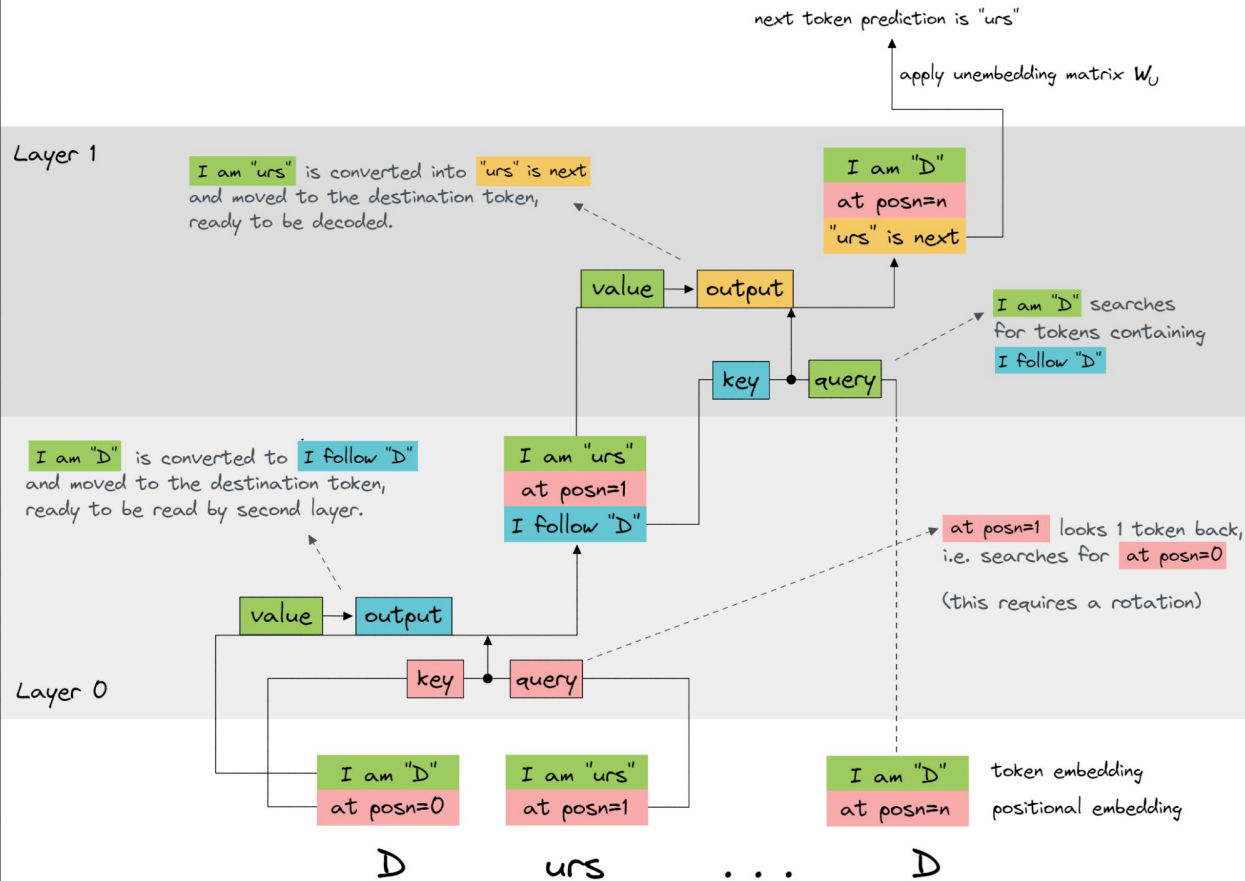
The OV circuit reads token information, and converts it into something which will result in a prediction of "urs".

We have:

$\begin{bmatrix} \text{I am "urs"} \\ \text{at posn=1} \\ \text{I follow "D"} \end{bmatrix}^T W_V W_O = \text{"urs" is next}$

which gets added to the residual stream of the second "D" token.

Putting everything together:



In-context Learning and Induction Heads

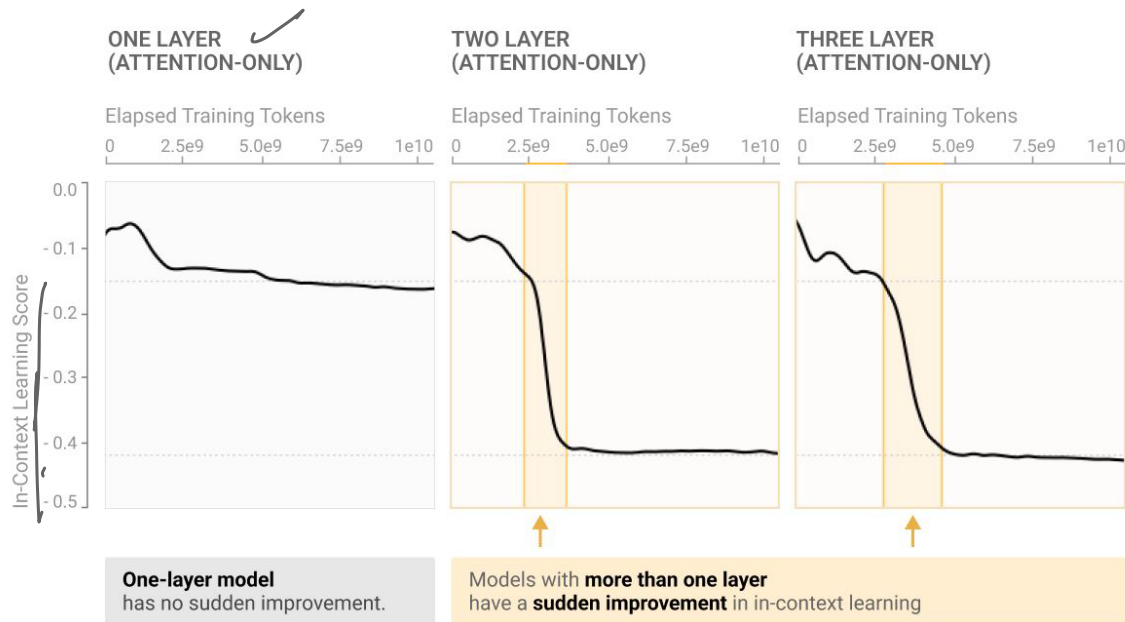
Argument 1 (*Macroscopic co-occurrence*): Transformer language models undergo a “phase change” early in training, during which induction heads form and simultaneously in-context learning improves dramatically.

Argument 2 (*Macroscopic co-perturbation*): When we change the transformer architecture in a way that shifts whether induction heads can form (and when), the dramatic improvement in in-context learning shifts in a precisely matching way.

Argument 3 (*Direct ablation*): When we directly “knock out” induction heads at test-time in small models, the amount of in-context learning greatly decreases.

Argument 4 (*Specific examples of induction head generality*): Although we define induction heads very narrowly in terms of copying literal sequences, we empirically observe that these same heads also appear to implement more sophisticated types of in-context learning, including highly abstract behaviors, making it plausible they explain a large fraction of in-context learning.

MODELS WITH MORE THAN ONE LAYER HAVE AN ABRUPT IMPROVEMENT IN IN-CONTEXT LEARNING



$$w_1 \quad w_{500} \quad w_{512}$$

$$\uparrow \quad \uparrow$$

$$\text{loss}(50) \quad \text{loss}(500)$$

We highlight the "phase change" period of training in plots to make visual comparison between plots easier. The highlighted region is selected for each model based on the derivative of in-context learning.

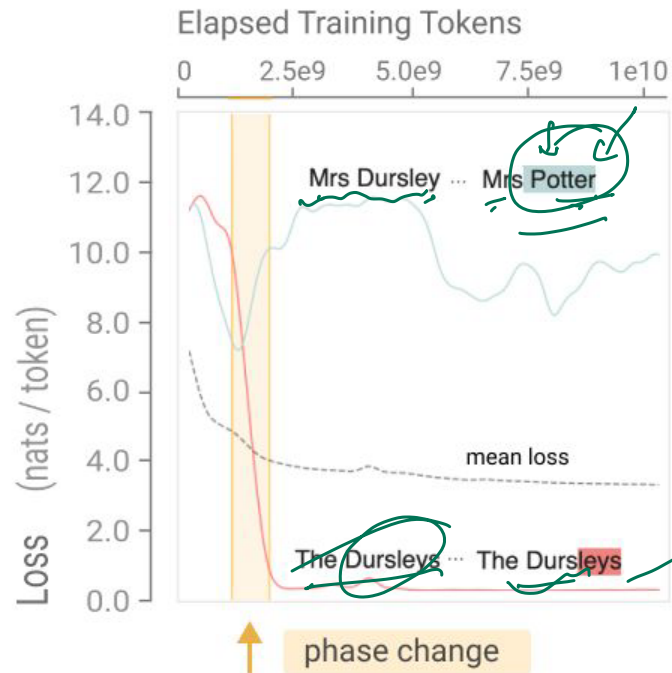
In-context learning score: the loss of the 500th token in the context minus the average loss of the 50th token in the context, averaged over dataset examples.

$$\text{loss}(500) - \text{loss}(50)$$

Why this choice? The 500th token is near the end of a length-512 context, and the 50th token is far enough into the context that some basic properties of the text have been established (such as language and document type) while still being near the start.

PER-TOKEN LOSSES OVER TRAINING

We look at two specific tokens in the Harry Potter passage a to highlight their movement during the phase change.



Mrs Potter : This token's loss increases during the phase change, because the model incorrectly predicts that the next token should be "D" from "Dursley".

The Dursleys : This token's loss decreases during the phase change, because the model correctly predicts "leys" based on a previous example.

In-context Learning and Induction Heads

Argument 5 (*Mechanistic plausibility of induction head generality*): For small models, we can explain mechanistically how induction heads work, and can show they contribute to in-context learning. Furthermore, the actual mechanism of operation suggests natural ways in which it could be repurposed to perform more general in-context learning.

Argument 6 (*Continuity from small to large models*): In the previous arguments, the case for induction heads explaining in-context learning is stronger for small models than for large ones. However, many behaviors and data related to both induction heads and in-context learning are smoothly continuous from small to large models, suggesting the simplest explanation is that mechanisms are the same.